

JNX-OS Quick Reference

Fast lookup for common operations and patterns.

Authentication

Server-Side (Page/API Route):

```
import { requireAuth, requireAdmin } from '@lib/auth/helpers';

// Require any authenticated user
const { user, jnxUser } = await requireAuth();

// Require admin role
const { user, jnxUser } = await requireAdmin();
```

Client-Side (Component):

```
import { useUser, useClerk } from '@clerk/nextjs';
import { useIsAdmin } from '@lib/auth/clerk-client';

const { user, isLoaded } = useUser();
const { signOut } = useClerk();
const isAdmin = useIsAdmin();
```

Database Operations

Idempotent User/Org Operations:

```
import {
  upsertUser,
  upsertOrg,
  createUserWithOrg,
  syncUserFromClerk
} from '@lib/db/helpers';

// Update or create user
await upsertUser({
  clerkUserId: 'user_xxx',
  email: 'user@example.com',
  fullName: 'John Doe',
  role: 'member'
});

// Create user with default org (transactional)
await createUserWithOrg({
  clerkUserId: 'user_xxx',
  email: 'user@example.com',
  fullName: 'John Doe'
});
```

Supabase Query:

```
import { createSupabaseServerClient } from '@lib/supabase/server';

const supabase = createSupabaseServerClient();

const { data, error } = await supabase
  .from('your_table')
  .select('*')
  .eq('user_id', userId)
  .is('deleted_at', null); // Respect soft-deletes
```

Security

Rate Limiting:

```
import { applyRateLimit } from '@lib/security/rate-limit';

const identifier = req.ip || 'anonymous';
const result = await applyRateLimit('auth', identifier);

if (!result.success) {
  return NextResponse.json(
    { error: 'Too many requests' },
    { status: 429 }
  );
}
```

Logging with PII Redaction:

```
import { Logger } from '@lib/observability/logger';

const logger = new Logger('api/your-route');

logger.info('User action', { userId: 'user_123' });
logger.error('Operation failed', { error });
```



UI Components

Buttons:

```
import { ButtonPrimary, ButtonSecondary } from '@components/ui';

<ButtonPrimary size="lg" glow onClick={handleClick}>
  Primary Action
</ButtonPrimary>

<ButtonSecondary size="md" onClick={handleClick}>
  Secondary Action
</ButtonSecondary>
```

Form Inputs:

```
import { InputField } from '@components/ui/input-field';
import { Mail } from 'lucide-react';

<InputField
  type="email"
  placeholder="Enter email"
  icon={<Mail className="h-5 w-5" />}
  onChange={handleChange}
/>
```

Status Badge:

```
import { StatusBadge } from '@components/ui/status-badge';

<StatusBadge status="connected">Clerk</StatusBadge>
<StatusBadge status="degraded">Supabase</StatusBadge>
```

API Routes

Standard Template:

```
import { NextRequest, NextResponse } from 'next/server';
import { currentUser } from '@clerk/nextjs/server';
import { Logger } from '@lib/observability/logger';

const logger = new Logger('api/your-route');

export async function POST(req: NextRequest) {
  try {
    // 1. Auth check
    const user = await currentUser();
    if (!user) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    // 2. Parse input
    const body = await req.json();

    // 3. Business logic
    // ...

    // 4. Success response
    return NextResponse.json({ success: true });
  } catch (error) {
    logger.error('Request failed', { error });
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

Testing Commands

```
# Build check
cd nextjs_space && yarn build

# Type check
cd nextjs_space && yarn tsc --noEmit

# Start dev server
cd nextjs_space && yarn dev

# Generate Prisma client
cd nextjs_space && yarn prisma generate

# View database
cd nextjs_space && yarn prisma studio
```

Common Issues & Solutions

Issue: “Column does not exist”

Solution: Run `MIGRATION_SIMPLE.sql` in Supabase SQL Editor

Issue: “Webhook failed”

Solution: Check Clerk webhook logs, verify `CLERK_WEBHOOK_SECRET`

Issue: “Dashboard shows setup screen forever”

Solution: Check Supabase connection, verify user exists in `users` table

Issue: “Admin panel shows ‘Unauthorized’”

Solution: Set user role in Clerk Dashboard → User → Metadata → Public:

```
{ "role": "admin" }
```

Issue: “Build fails with module not found”

Solution: Run `yarn install && yarn prisma generate`

Important File Locations

Configuration:

<code>.env</code>	# Environment variables
<code>next.config.js</code>	# Next.js config
<code>tailwind.config.ts</code>	# Design system
<code>middleware.ts</code>	# Route protection

Authentication:

<code>lib/auth/helpers.ts</code>	# requireAuth, requireAdmin
<code>lib/auth/clerk-client.ts</code>	# Client hooks
<code>lib/auth/clerk-server.ts</code>	# Server utilities
<code>lib/auth/rbac.ts</code>	# Roles & permissions

Database:

<code>lib/db/schema-v2.sql</code>	# Current schema
<code>lib/db/helpers.ts</code>	# UPSERT functions
<code>MIGRATION_SIMPLE.sql</code>	# GDPR migration

Security:

<code>lib/security/rate-limit.ts</code>	# Rate limiting
<code>lib/security/headers.ts</code>	# Security headers
<code>lib/privacy/redaction.ts</code>	# PII redaction

UI:

<code>components/ui/*</code>	# JNX Dark components
<code>app/globals.css</code>	# Global styles

API:

<code>app/api/webhooks/clerk/</code>	# Clerk sync webhook
<code>app/api/system/health/</code>	# Health metrics

Environment Variables

```
# Clerk (Authentication)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_xxx
CLERK_SECRET_KEY=sk_test_xxx
CLERK_WEBHOOK_SECRET=whsec_xxx

# Supabase (Database)
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJxxx
SUPABASE_SERVICE_ROLE_KEY=eyJxxx

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Deployment Checklist

- ☐ Run `yarn build` - must succeed
 - ☐ Verify `.env` variables in Vercel
 - ☐ Run `MIGRATION_SIMPLE.sql` in production Supabase
 - ☐ Configure Clerk webhook URL (production)
 - ☐ Test signup/login flows
 - ☐ Verify admin access
 - ☐ Check Vercel deployment logs
 - ☐ Monitor Clerk webhook success rate
-

Need more details? Check `BACKEND_PROTECTION_PROMPT.md` 